Choix de la taille de cluster FAT32 et organisation des fichiers de mesures

Projet système embarqué : enregistrement des mesures sur carte microSD

1. Contexte et hypothèses

- Support : carte microSD de 8 Go formatée en FAT32, secteurs de 512 octets. Le secteur est imposé par le matériel, alors que la taille du cluster se choisit au formatage.
- Une ligne de mesure fait au maximum 256 octets.
- Intervalle entre deux mesures : 10 min par défaut (paramètre LOG_INTERVAL). Cette valeur est une hypothèse du projet, tirée du document « Modes de fonctionnement » et non de l'énoncé du prosit.
- Organisation : un répertoire par jour.

Calculs de base :

- Mesures par jour : 24 × 60 / 10 = 144 lignes
- Volume maximal par jour : 144 lignes × 256 octets (taille maximale d'une ligne) = 36 864 octets = 36 Kio
- Volume maximal par heure : 6 × 256 = 1 536 octets = 1,5 Kio

2. Rappels sur FAT32

Un fichier occupe toujours un nombre entier de clusters :

nombre de clusters = ⌈taille du fichier / taille d'un cluster⌉ (arrondi à l'entier supérieur, car le dernier cluster n'est souvent que partiellement rempli)

espace alloué = nombre de clusters × taille d'un cluster

Exemple : un fichier de 1 536 octets avec des clusters de 1 024 octets (1 Kio) :

- nombre de clusters = ⌈1 536 / 1 024⌉ = ⌈1,5⌉ = 2
- espace alloué = 2 × 1 024 = 2 048 octets

Le gaspillage (« slack ») est la différence entre l'espace alloué et la taille réelle. Il est au maximum d'un cluster moins un octet par fichier, et en moyenne d'un demi-cluster.

La FAT est un tableau d'entrées de 4 octets, une par cluster, qui indique le cluster suivant. Pour un fichier occupant les clusters 5, 6 et 9 : FAT[5] = 6, FAT[6] = 9, FAT[9] = fin de chaîne. **C'est donc le fichier lui-même qui est une liste chaînée de clusters**, et non l'ensemble des fichiers.

Un répertoire est un fichier composé d'entrées de 32 octets (nom, attributs, dates, taille, premier cluster). Trouver un fichier revient à parcourir les entrées du répertoire, puis lire un fichier revient à suivre sa chaîne de clusters dans la FAT.

La « liste chaînée de fichiers » évoquée dans le sujet est une structure que l'on construirait soi-même au-dessus de FAT32 (chaque fichier pointant vers le suivant). Ce n'est pas ce que FAT32 fait nativement.

3. Point de départ : clusters de 64 Kio

L'idée initiale est de prendre des clusters de 64 Kio pour « accélérer les transferts ». Cela se justifie pour de gros fichiers lus séquentiellement, mais pas pour de petits fichiers de log.

3.1 Un seul fichier par jour

Grandeur	Calcul	Résultat
Espace alloué par jour	1 cluster de 64 Kio	64 Kio
Données utiles	144 × 256 octets	36 Kio
Taux de remplissage	36 / 64	0,5625 (56 %)
Perte	64 − 36	28 Kio (≈ 44 %)
Consommation annuelle	64 Kio × 365	23 360 Kio ≈ 22,8 Mio
Autonomie sur 8 Go	8×10^9 / (64 × 1024 × 365)	≈ 334 ans

On perd environ la moitié de l'espace, mais en valeur absolue c'est négligeable : la capacité n'est pas le facteur limitant.

3.2 Un fichier par heure (24 fichiers par jour)

Grandeur	Calcul	Résultat
Espace alloué par jour	24 × 64 Kio	1 536 Kio
Données utiles	24 × 1,5 Kio	36 Kio
Perte	1 536 − 36	1 500 Kio (≈ 97,7 %)
Surconsommation	1 536 / 36	≈ 42,7 ×
Consommation annuelle	1 536 Kio × 365	560 640 Kio ≈ 547 Mio
Autonomie sur 8 Go	8×10^9 / (560 640 × 1024)	≈ 14 ans

Chaque fichier occupe au minimum un cluster entier : avec 64 Kio, un fichier de 1,5 Kio gaspille presque tout son cluster. Cette organisation est donc très coûteuse.

4. Pourquoi un seul fichier par jour pose problème : la recherche

Lignes de longueur variable. Avec des lignes de longueur fixe (256 octets), l'offset d'une mesure se calcule directement : la 50e ligne commence à (50 − 1) × 256 = 12 544 octets. Avec des lignes de longueur variable, aucun calcul ne donne l'offset d'un horodatage.

Recherche dichotomique dans le fichier. On déplace le curseur de lecture (seek) à un offset estimé, on se resynchronise sur le début de la ligne suivante, on lit l'horodatage, puis on recommence sur la moitié concernée. Pour 144 lignes, cela demande au plus $\lceil \log_2 144 \rceil = 8$ itérations.

Chaîne de clusters. Avec des clusters de 1 Kio, un fichier de 36 Kio est une chaîne de 36 clusters : un seek peut devoir parcourir jusqu'à 36 entrées de FAT (complexité linéaire). Avec des clusters de 64 Kio, c'est un seul cluster. L'effet reste mineur.

Robustesse. La taille du fichier n'est mise à jour dans l'entrée de répertoire qu'à la fermeture ou à la synchronisation. Une coupure d'alimentation peut faire perdre une journée entière si on ne synchronise pas régulièrement, alors qu'avec des fichiers horaires la perte est limitée à une heure.

Organisation	Recherche de 14 h 00
1 fichier par jour	seek + resynchronisation, jusqu'à 8 itérations, complexité O(log n)
24 fichiers par jour	ouverture directe du fichier de l'heure (le nom sert d'index), lecture d'au plus 6 lignes, complexité O(1)

La taille de cluster change peu la difficulté de recherche : c'est l'organisation en fichiers qui la détermine.

5. Solution retenue : fichiers horaires et clusters de 1 Kio

Un fichier horaire contient au plus 6 × 256 = 1 536 octets.

Taille de cluster	Clusters par fichier	Alloué par fichier	Perte par fichier	Perte (%)	Alloué par jour (24 fichiers)
64 Kio	1	65 536 octets	64 000 octets	97,7 %	1 536 Kio
4 Kio	1	4 096 octets	2 560 octets	62,5 %	96 Kio
1 Kio	2	2 048 octets	512 octets	25 %	48 Kio
512 octets	3	1 536 octets	0 octet	0 %	36 Kio

- 1 Kio utilise 1 536 / 48 = 32 fois moins d'espace que 64 Kio, et deux fois moins que 4 Kio.
- Consommation annuelle avec 1 Kio : 48 × 365 = 17 520 Kio ≈ 17,1 Mio, contre ≈ 34,2 Mio avec 4 Kio.
- Le paramètre FILE_MAX_SIZE, hypothèse du projet tirée du document « Modes de fonctionnement » (et non de l'énoncé du prosit), est de 2 Ko par défaut : 2 048 / 1 024 = 2 clusters de 1 Kio exactement, donc aucune perte quand le fichier est plein. Avec des clusters de 4 Kio, ce même fichier n'en remplirait que 2 048 / 4 096 = 50 %. Pour 4 096 octets, on obtient 4 clusters de 1 Kio exactement.
- 256 octets est un maximum : avec des lignes plus courtes, la perte reste bornée à un cluster par fichier, et un petit cluster limite cette perte.

6. Coût de la FAT

Le nombre de clusters vaut 8×10^9 / taille de cluster. La FAT occupe 4 octets par cluster et existe en deux copies.

Taille de cluster	Nombre de clusters	FAT (1 copie)	FAT (2 copies)	Part de la carte
64 Kio	≈ 122 070	≈ 0,49 Mo	≈ 1 Mo	≈ 0,01 %
4 Kio	≈ 1 953 125	≈ 7,8 Mo	≈ 15,6 Mo	≈ 0,2 %
1 Kio	≈ 7 812 500	≈ 31,3 Mo	≈ 62,5 Mo	≈ 0,78 %
512 octets	≈ 15 625 000	≈ 62,5 Mo	≈ 125 Mo	≈ 1,56 %

Des clusters de 512 octets supprimeraient la perte dans notre cas, mais doublent la taille de la FAT pour un gain minime. Avec 1 Kio, la FAT reste sous 1 % de la carte : c'est un bon compromis. Une FAT plus grande signifie aussi plus d'entrées à parcourir et à mettre en cache sur un microcontrôleur à mémoire limitée.

7. Métadonnées et aide à la recherche

Le nommage 200531_0.LOG, tiré du document « Modes de fonctionnement » (et non de l'énoncé du prosit) et retenu comme hypothèse du projet, encode la date et un numéro de révision, mais pas l'heure. Plusieurs options permettent de retrouver rapidement le bon fichier :

1. **Nommer les fichiers par heure.** Le nom devient l'index. Une entrée de répertoire fait 32 octets : un dossier de 24 fichiers avec les entrées « . » et « .. » occupe 26 × 32 = 832 octets, soit un seul cluster de 1 Kio (noms au format 8.3).
2. **Ajouter une ligne d'en-tête** avec l'horodatage de début de fichier (≈ 20 octets). Le fichier passe de 1 536 à 1 556 octets, ce qui reste dans 2 clusters (2 048 octets) : le coût en espace est nul, car l'en-tête se glisse dans les 512 octets déjà perdus.
3. **Tenir un fichier index par jour** listant, pour chaque fichier, ses horodatages de début et de fin : 24 lignes × ≈ 30 octets = 720 octets, soit un cluster de 1 Kio par jour (≈ 365 Kio par an). On évite d'ouvrir les fichiers un par un.

Si l'on garde la rotation par taille décrite dans le document « Modes de fonctionnement » (FILE_MAX_SIZE = 2 Ko, hypothèse du projet absente de l'énoncé du prosit), on obtient au maximum 36 864 / 2 048 = 18 fichiers par jour. Un parcours linéaire (comme une liste chaînée) demande jusqu'à 18 ouvertures, alors qu'une recherche dichotomique sur les numéros de révision, avec l'horodatage de début en en-tête, n'en demande que $\lceil \log_2 18 \rceil = 5$.

On échange ainsi un peu d'espace (quelques dizaines d'octets par fichier) contre une recherche beaucoup plus rapide, et ce surcoût est pratiquement nul grâce à la perte déjà présente dans le dernier cluster.

8. Sensibilité à l'intervalle de mesure

Si LOG_INTERVAL passe à 1 min, un fichier horaire fait 60 × 256 = 15 360 octets = 15 Kio.

Taille de cluster	Clusters par fichier	Alloué	Perte
64 Kio	1	64 Kio	49 Kio (≈ 76,6 %)
4 Kio	4	16 Kio	1 Kio (6,25 %)
1 Kio	15	15 Kio	0

La taille de cluster optimale dépend donc de l'intervalle et de la taille de fichier choisis. Le choix de 1 Kio reste robuste sur toute la plage configurable, alors que 64 Kio est mauvais dans tous les cas où les fichiers sont petits.

9. Conclusion

- Les clusters de 64 Kio proposés au départ ne sont « pas trop gaspilleurs » (≈ 44 %) que si l'on écrit un seul fichier par jour. Dans ce cas, l'espace n'est de toute façon pas un problème : plus de 300 ans d'autonomie sur 8 Go.
- Ce choix rend la recherche difficile : lignes de longueur variable, dichotomie avec resynchronisation, et risque de perte d'une journée entière en cas de coupure.
- Subdiviser en 24 fichiers horaires simplifie la recherche (le nom du fichier sert d'index), mais avec des clusters de 64 Kio la perte atteint ≈ 98 % et on consomme 42,7 fois plus d'espace.
- Des clusters de 1 Kio avec 24 fichiers horaires donnent la meilleure combinaison : perte de 25 % seulement (48 Kio par jour au lieu de 1 536 Kio), recherche directe par le nom de fichier, et FAT de seulement 0,78 % de la carte.
- Quelques octets de métadonnées (en-tête ou index) améliorent encore la recherche sans coût notable en espace.

Formatage sous Linux : `sudo mkfs.fat -F 32 -s 2 /dev/sdX1` *(2 secteurs × 512 octets = clusters de 1 Kio), à vérifier avec* `fsck.fat -v`*.*